

ACCESSIBILITY

Rechtlich: UNO-Behindertenrechtskonvention (UNO-BRK), gleichberechtigte Möglichkeiten im Zugang & Benutzung von Informations- & Kommunikationssystemen
Häufigste Fehler: Text zu wenig Kontrast, kein alt Text, leere Links/Buttons/Doc-Sprache, fehlende Form Labels
Farbenblindheit: prüfen mit Dev-Tools (Rendering), Doppel-Codierung notwendig (Farbe & Form/icon/Text), kritische Informationen nicht nur mit Text codieren, farbenblind-freundliche Farbpaletten wählen
Farbkontrast: prüfen mit Dev-Tools (Color Picker) & Lighthouse / Firefox Report, Zielgruppe erweitern mit WCAG Kontrast Level AA (5.7:1, 50+), AAA (15.9:1, 80+) Zoombarkeit: Zoom nicht unterbinden, Falsch: `user-scalable=0`/no, maximum-scale=1
Animationen: sollten abstellbar sein, Verringerung der Ablenkung bei Konzentrationsstörung, Verhinderung der Auslösung von Epilepsie und Migräne
Tastatur-Bediensbarkeit: alle wichtigen Input-Elemente sind in der richtigen Reihenfolge fokussierbar, verboten: float, flex-direction
-reverse, Standard Input Controls nutzen: `a, button, input[type=„...“], textarea`, zusätzliche Elemente können mit `«tabindex=0»` fokussierbar gemacht werden, Elemente jeder Seite können mit `«autofocus»` versehen werden
Screenreader-Optimierung: keine Headings Level auslassen, semantische Elemente richtig nutzen, Skip-Links am Anfang der Site, lang Attribut korrekt setzen
ARIA-Attributes: lösen Accessibility Issues, die nicht mit nativem HTML gelöst werden können, heissen jeweils aria-, wichtigste: hidden, label, expanded, current, labelledby, controls, haspopup, invalid, live, required
Tabellen: Heading für Rows & Columns mit `<th>` ausstatten, alternativ `<td role=“columnheader/rowheader“>`
Allgemeines: alt Tag bei Bildern, leer wenn dekorativ, logische Reihenfolge auch ohne CSS, semantische Struktur, Multimedia: 2-Sinne Prinzip, Anzeige-Art kann angepasst werden (Textgrösse, Animationen, Zeitlimits)
Überschriften/Form Labels/Linktexte sind verständlich, auf Formatwechsel (z.B. PDF) wird hingewiesen, konsistente Navigation, valides HTML, kompatibel mit versch. I/O Geräten, Hilfestellung bei Interaktionen, PDFs auf PDF

SECURITY

Stored XSS: Server kann so manipuliert werden, dass JS Code an die Browser der Opfer ausgeliefert & ausgeführt wird, Massnahmen: User-Input muss beim Output encodet werden, z.B. mit `3x {}` in Handlebars, oder User-Input wird sanitized, z.B. mit xss oder dompurify
Library JS injection: dem Server kann JS Code gesendet werden, welcher ihn im Code mit `eval()`/`setTimeout()`/`setInterval()`/`Function()` ausführt, Massnahmen: stattdessen `parseXXX()`/`JSON.parse()` verwenden, globale Scopes & Variablen reduzieren, rechen-intensive Tasks mit `childprocess.spawn` auslagern, Node kein Root-Prozess
Insecure direct object references: durch Manipulation der URL lassen sich sensible Daten ohne Authentifizierung oder Autorisierung aufrufen, Massnahmen: bei allen Sites mit sensiblen Daten sicherstellen, dass der User eingeloggt und berechtigt ist

CSRF

Cross Site Request Forgery: Angreifer bringt Nutzer mit gültiger Session dazu, auf einer Site ein gefälschtes Formular an die richtige Site zu submitten, Massnahmen: Formular-Submission-Requests (GET/POST) müssen geprüft werden, ob sie von einem vom Server frisch ausgelieferten Formular stammen (mit CSRF Token)
Replay attacks: z.B. bei Quiz wird mit Formular Aufgabe und User-Input-Lösung submitted, wenn richtig gibt es einen Punkt, das kann replayed werden, sollte aber nicht gehen, da der User dann unrechtmässig Punkte erhält, Massnahmen: CSRF Token 1x gültig, bereits gelöste Aufgaben speichern, letzte Aufgabe in Session speichern
Cryptographic failures: Crypto schlecht eingesetzt, Massnahmen: https nutzen, keine geheimen Infos in Query-Params (Leaks in Server Logs oder Browser History), Authentication Service nutzen
Identification & Authentication failures: z.B. ist Session Timeout zu lang, User bleibt auf öffentlichem PC eingeloggt, Massnahme: Session Timeout sinnvoll setzen
CORS Header: ermöglicht Laden von Ressourcen (Bilder, Scripts, usw.) von anderem Ursprungsort als eigener Origin, man beschränkt, welche Websites auf seine Res-sourcen zugreifen dürfen

CSP Header: Content Security Policy, Beispiel schränkt Kommunikation auf Source Domain/api.ch ein, externe & Inline-Scripts sind verboten, aktivieren mit unsafe-inline
Weiteres: DNS Prefetching & Client Caching deaktivieren, Public Key Pinning Headers, Strict-Transport-Security Header (HSTS) setzen, X-XSS-Protection Header, Safe-Regex verwenden (DDoS), Parameter pollution
Cookies: Secure Attribut: Cookie wird nur über HTTPS versendet, nicht HTTP (ausser localhost), HttpOnly Attribut: deaktiviert Zugriff auf Cookies aus JS